findings

CodeGraph codegraph_search vs codegraph_explore — Root-Cause Findings

Date: 2026-06-22 | Author: systematic-debugging subagent (HelixCode) Scope: Why codegraph_search returns “No results found” for “provider Generate LLM” while codegraph_explore resolves the same intent. Iron Law applied (§11.4.6/§11.4.102/§11.4.123).

Environment (FACT): codegraph v1.0.1 (/Users/milosvasic/.local/bin/codegraph -> /opt/homebrew/bin/codegraph, npm @colbymchenry/codegraph, native @colbymchenry/codegraph-darwin-arm64). Index .codegraph/codegraph.db ~4.3GB. codegraph_validate.sh = 18/18 PASS. MCP wired in .mcp.json.

1. Tool contract (verbatim from live MCP schema)

codegraph_search: “Quick symbol search by name. Returns locations only (no code). Use codegraph_explore instead to get the actual source / understand an area in one call.” query = “Symbol name or partial name (e.g., "auth", "signIn", "UserService")”. codegraph_explore: “PRIMARY TOOL … Query can be a natural-language question OR a bag of symbol/file names.” CLI: `query <search>` = “Search for symbols”; `explore <query...>` = area/source+call-paths. Reading: search = symbol-NAME/partial lookup (examples are all single tokens); explore = NL or symbol bag.

2. Query matrix (verbatim codegraph_search MCP results)

a1 “OllamaProvider” -> 10 results WORKS (single exact symbol) a2 “NewOllamaProvider” -> 10 results WORKS a3 “CrushGenerator” -> 6 results WORKS b1 “Ollama” -> 10 results WORKS (single substring token) b2 “Generate” (limit 3) -> 3 results WORKS d1 “ollama_provider.go” -> 3 results WORKS (file fragment) c1 “provider Generate LLM” -> No results found FAILS (the reported case) c2 “list models from providers”-> No results found FAILS (NL phrase) c3 “provider Generate” -> No results found FAILS (2 tokens, no co-occurring symbol) c4 “Provider Generate” k=method-> No results found FAILS e1 “Ollama provider”

-> 10 results (top OllamaProvider) WORKS (tokens co-occur in one symbol) e2 “OllamaProvider Generate” -> 10 results WORKS e3 “Generate LLM” -> 10 results (top generateLLM @ helix_code/internal/server/llm_generate.go:179) WORKS

Boundary is NOT “single vs multi-word”. Multi-word WORKS when all tokens co-occur within ONE indexed symbol’s FTS columns (e1 OllamaProvider has Ollama+provider; e3 generateLLM has Generate+LLM). It FAILS when no single symbol contains ALL tokens (c1/c3).

3. Index mechanism (FACT: sqlite3 .codegraph/codegraph.db schema)

CREATE VIRTUAL TABLE nodes_fts USING fts5(id, name, qualified_name, docstring, signature, content=‘nodes’, content_rowid=‘rowid’) – plus nodes_ai/_ad/_au sync triggers. => codegraph_search is a SQLite FTS5 virtual table over 5 columns. FTS5 default operator between bare terms is AND: MATCH ‘provider Generate LLM’ needs ONE row (symbol) whose columns contain all 3 tokens. None does -> zero rows -> “No results found”. Exactly reproduces the matrix (c1/c3 = cross-symbol AND impossible in one row; e1/e3 = both tokens in one symbol name). (Host android-platform-tools sqlite3 lacks fts5 module -> direct MATCH probe errored “no such module: fts5”; codegraph ships its own fts5-enabled sqlite, which is why the MCP tool works. The §2 matrix is the proof; §3 schema is the mechanism.)

4. Web research (>=2 angles, access date 2026-06-22)

- github.com/colbymchenry/codegraph + colbymchenry.github.io/codegraph/getting-started/introduction/: tree-sitter -> SQLite with FTS5 full-text search; explore is PRIMARY (NL question OR symbol bag); search is the symbol full-text surface; docs do NOT describe search as semantic/NL.
- tosea.ai/blog/codegraph-claude-code-cursor-guide-2026: “codegraph_explore … bag of symbol names spanning the flow” — positions explore (not search) as the area/NL tool.
- npmjs.com/package/@colbymchenry/codegraph: version 1.0.1 (confirmed via codegraph –version; page 403 on automated fetch).
- Issue #850 (cited in daemon log “Main thread unresponsive ~60s — killing wedged process”) = GitHub issue #850 “codegraph cause 100% CPU”, closed completed 2026-06-13 — a CPU/watchdog concern, UNRELATED to search.
- Negative finding: no upstream issue documents multi-word-empty search; no doc warns “search is implicit-AND”.

5. FACT VERDICT

WORKING-AS-DESIGNED with a documentation/affordance gap — NOT a code bug, NOT an index defect. codegraph_search is an FTS5 symbol-name lookup over {name,qualified_name,docstring,signature} with FTS5 default implicit-AND. “provider Generate LLM” returns nothing because no single symbol contains all 3 tokens — the correct result for an AND full-text query. The tool contract says “symbol name or partial name” and shows only single-token examples; codegraph_explore is the documented NL/multi-concept surface (it returned 205 symbols/157 files for the same intent this session). The only real defect is missing AGENT GUIDANCE on when to use search vs explore. Not a §11.4 false-success; index is healthy (18/18 PASS; single-symbol search perfect).

6. WIDEST-LEVEL REMEDIATION (recommendation only — no files edited)

6.1 PRIMARY (constitution §11.4.78): add a “search vs explore” rule to constitution/docs/codegraph/* (e.g. Status.md or new TOOL_SELECTION.md, inherited by reference per §11.4.80). Proposed text: “codegraph_search is a SQLite FTS5 symbol-name lookup (name, qualified_name, docstring, signature) with implicit-AND between terms — it matches ONE symbol whose indexed fields contain ALL your tokens. Use it ONLY when you can name a specific/partial symbol (OllamaProvider, NewOllamaProvider, Ollama). A multi-word natural-language phrase (provider Generate LLM, list models from providers) will correctly return ‘No results found’ because no single symbol contains every token — expected FTS5 behavior, NOT a broken index. For any NL question / area survey / multi-concept query use codegraph_explore (PRIMARY; accepts NL or symbol bag; returns source + call paths in one call). Rule of thumb: name a symbol -> codegraph_search; ask a question -> codegraph_explore. An empty codegraph_search is a signal to re-issue via codegraph_explore, never that the index is broken (confirm health with codegraph_validate.sh).” Mirror into AGENT_GUARDRAILS preamble (§11.4.109) so every dispatched subagent gets it; cascade the §11.4.78 note to CLAUDE.md/AGENTS.md/QWEN.md/GEMINI.md per §11.4.157 lockstep. 6.2 SECONDARY (optional, upstream §11.4.74): file upstream issue/PR on colbymchenry/codegraph so that when codegraph_search gets 0 AND-rows for a multi-token query it either retries with OR / per-term prefix term* (labeled), or returns a hint “0 exact-AND matches for N tokens — try one symbol name or codegraph_explore”. Extend the catalogue tool upstream rather than fork. OPTIONAL — 6.1 fully fixes the agent-facing problem. 6.3 Do NOT: re-index (healthy), or treat #850 (100% CPU) as related.

7. Evidence index

§1 live MCP ToolSearch load of codegraph_{search,explore,node,callers}; §2 13 live codegraph_search calls (verbatim); §3 sqlite3 master dump of nodes_fts (fts5) + triggers; §4 4 cited sources + issues page (2026-06-22); version codegraph –version=1.0.1.